

- 01 Step 0 — 환경 진단 (git/gh 추가)
- 02 Step 1 — CLAUDE.md 협업 작성 + AI 검증 루프
- 03 Step 2 — PRD 참조 + docs.rules 환경 구축
- 04 Step 3 — settings.json + Permissions
- 05 Step 4 — GitHub 저장소 셋업 + 초기 커밋
- 06 Step N — 정리와 비용 비교
- 07 과제 + Tips 6

— STEP 0

환경 진단

환경 확인과 토큰 베이스라인 측정 + git/gh 추가

- 01 0-1. 환경 확인 명령 (node·claude·git·gh)
- 02 0-2. 작업 폴더 + git 초기화 (planning_project)
- 03 0-3. 토큰 베이스라인 + PRD 복사 (S3 docs/prd.md)

— 활동 0-1 · 산출물: 4 도구 확인 완료

환경 확인 명령 — node·claude·git·gh

통과 조건: node 18.0 이상, claude·git·gh 모두 동작.

MACOS / LINUX

```
node --version
claude --version
git --version
gh --version
```

WINDOWS POWERSHELL

```
node --version
claude --version
git --version
gh --version
```


— 활동 0-2 · 산출물: PLANNING_PROJECT + GIT 저장소 초기화

작업 폴더 + git 초기화

planning_project 고정 — S3에서 누적된 폴더 위에 git을 초기화한다.

MACOS / LINUX

```
mkdir planning_project  
cd planning_project  
git init
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Path planning_project  
Set-Location planning_project  
git init
```

— 활동 0-3 · 산출물: 베이스라인 토큰 + DOCS/PRD.MD

토큰 베이스라인 + PRD 복사

시작 토큰을 메모. 마지막 Step에서 비교. PRD는 S3에서 만든 prd.md를 복사.

CLAUDE CODE 세션 안에서

/cost

PRD 복사 (S3 산출물)

```
mkdir -p docs
cp ../planning_project/docs/prd.md docs/
# 또는 같은 폴더면 생략
```

STEP 0 · 검증 / 성공 기준

Step 1 진입 가능한가

5개 항목 모두 통과 시 Step 1 (CLAUDE.md 협업 작성)로 진입.

- 01

☒

폴더 이름이 planning_project인가
- 02

☒

git 저장소 초기화됐는가
- 03

☒

docs/prd.md가 작업 폴더에 있는가
- 04

☒

/cost 베이스라인을 메모했는가
- 05

☒

gh 명령이 동작하는가

— STEP 1

CLAUDE.md 협업 작성과 AI 검증 루프

작업자 1차 초안 + AI 개선 + 검증 통과

- 01 1-1. 빈 CLAUDE.md 생성 → 1차 초안 (작업자 손)
- 02 1-2. AI 개선 요청 (CLAUDE.md + PRD 참조)
- 03 1-3. 작업자 수정 (수용 / 거부 / 부분 적용)
- 04 1-4. AI 검증 루프 (의도 일치까지 반복)
- 05 1-5. 분량 점검 (200줄 미만)

활동 1-1 · 산출물: 빈 CLAUDE.MD

빈 CLAUDE.md 생성

planning_project/ 루트에 CLAUDE.md 파일을 만든다.

MACOS / LINUX

touch CLAUDE.md

WINDOWS POWERSHELL

New-Item -ItemType File -Path CLAUDE.md

1차 초안 — 빈 양식 + 시드 6 사례

완벽하지 않아도 된다 — 작업자 의도를 거칠게 담는 것이 목적.

빈 양식

CLAUDE.md

1. 프로젝트 개요

{한 단락: 페르소나 + 핵심 기능 3개 + 성공 지표}

2. 기술 스택과 파일 구조

{스택과 폴더 구조}

3. 코딩 가이드라인

{네이밍, 커밋, 테스트}

4. 제약사항

{하지 않을 것}

자세한 내용은 docs/prd.md 참조

시드 6번 거친 초안

CLAUDE.md

1. 프로젝트 개요

1인 스마트스토어 셀러를 위한 카피 생성 도구.

상품 사진과 키워드로 카피 3개를 빠르게 만들어줌.

2. 기술 스택

- Next.js + Tailwind

- OpenAI API

3. 코딩 가이드라인

- camelCase 함수

- 커밋은 feat/fix 형식

4. 제약사항

- 모바일 안 함

- 한국어만

자세한 내용은 docs/prd.md 참조

AI 개선 요청 — 명령 + 시드 6 답변

4영역 충분성 / 모호 표현 / PRD 핵심 누락 / 빠진 부분 — 4 기준 제시.

CLAUDE CODE 명령

"현재 작성된 CLAUDE.md를 읽고 docs/prd.md를 참조해서 개선해줘.
다음 기준으로 검토:
- 4영역이 모두 충분히 채워졌는가
- 모호한 표현이 있는가
- PRD의 핵심 정보가 빠지지 않았는가
- 빠진 부분이 있다면 어디인가
개선안과 함께 이유를 설명해줘."

시드 6번 AI 개선 답변

- 1영역 개선:
 - "빠르게" → "5초 이내에"
 - 페르소나 모호 → "월 매출 100-500만원 1인 셀러"
- 2영역 보강:
 - BE 스택 누락
 - 권장: "Node.js + Express, OpenAI GPT-4o"
- 3영역 보강:
 - 컴포넌트 네이밍 누락
 - 명확한 테스트 도구 권장
- 4영역 추가 권장:
 - "이미지 자동 생성 미지원"
 - "광고 자동 집행 미지원"
 - "결제 기능 미포함"

— 활동 1-3 · 산출물: 작업자 수정 후 CLAUDE.MD

작업자 수정 원칙 + 시드 6 1영역 정제

AI 제안을 그대로 받지 않는다. 3 원칙으로 선택적 적용.

수정 3 원칙

수용	AI가 PRD 기반 정확히 짚은 부분
거부	작업자가 의도적으로 뺀 내용
수정	AI 제안을 일부만 적용

시드 6번 1영역 정제 결과

월 매출 100-500만원 규모의
1인 스마트스토어 셀러를 위한
차별화 카피 자동 생성 도구.
상품 사진과 키워드 5개를 입력하면
카테고리별 차별화 카피 3개 후보를
5초 이내에 생성.
성공 지표: 카피 작성 시간 1-2시간 → 5분 이내,
카피 후보 사용률 70% 이상.

— 활동 1-4 · 산출물: AI 검증 통과

AI 검증 루프 — 명령 + 시드 6 통과

의도와 일치할 때까지 반복. 불일치 → 모호 영역 보강 → 재검증.

검증 명령

"이 CLAUDE.md만 보고 우리 프로젝트가
무엇인지 한 문장으로 요약해줘"

AI 답변 (시드 6)

"1인 스마트스토어 셀러를 위한
차별화 카피 자동 생성 도구.
상품 사진과 키워드를 입력하면
5초 이내에 카피 3개를 출력.
모바일은 지원 않고 한국어 웹 전용."

점검 결과

- OK 페르소나 일치
 - OK 핵심 기능 일치
 - OK 성공 지표 반영
 - OK 제약사항 반영
- 검증 통과

— 활동 1-5 · 산출물: 200줄 미만 확인

분량 점검 — 200줄 미만 균형

매 명령마다 자동 로드되는 토큰. 4영역 모두 채우면서 30-50줄이 권장.

MACOS / LINUX

wc -l CLAUDE.md

WINDOWS POWERSHELL

(Get-Content CLAUDE.md | Measure-Object -Line).Lines

— STEP 1 · 검증 / 성공 기준

Step 2 진입 가능한가

5개 항목 모두 통과 시 Step 2 (PRD 참조 + 환경 구축)로 진입.

01 ☒ 1차 초안이 작업자 손으로 작성됐는가

02 ☒ AI 개선 제안을 검토·수용·거부했는가

03 ☒ AI 검증 답변이 작업자 의도와 일치하는가

04 ☒ CLAUDE.md가 200줄 미만인가

05 ☒ 4영역이 모두 채워졌는가

— STEP 2

PRD 참조 지침과 초기 환경 구축

docs/와 rules/ 폴더 구축 + CLAUDE.md 참조 지침

- 01 2-1. docs/ 폴더 (prd / persona / painpoints)
- 02 2-2. rules/ 폴더 (coding-style / commit / architecture)
- 03 2-3. rules/ AI에 채우게 하기
- 04 2-4. CLAUDE.md 마지막에 시스템 설계 참조 지침 추가

docs/ 폴더 구성 + 페르소나·페인포인트 복사

S3에서 만든 페르소나·페인포인트를 docs/ 표준 위치로 복사.

표준 구성

```
docs/
|—— prd.md      ← S3 export
|—— persona.md  ← 페르소나 가설
└—— painpoints.md ← 페인포인트
```

복사 명령

MACOS / LINUX

```
cp ../planning_project/docs/persona.md docs/
cp ../planning_project/docs/painpoints.md docs/
```

WINDOWS POWERSHELL

```
Copy-Item "..\..\planning_project\docs\persona.md" docs/
Copy-Item "..\..\planning_project\docs\painpoints.md" docs/
```


rules/ 폴더 구성 + 빈 파일 생성

다음 단계에서 AI가 CLAUDE.md 3영역을 참고해 내용을 채운다.

표준 구성

```
rules/
|—— coding-style.md      ← 네이밍·포맷
|—— commit-convention.md ← 커밋 메시지
└—— architecture.md     ← 폴더 구조 원칙
```

생성 명령

MACOS / LINUX

```
mkdir -p rules
touch rules/coding-style.md
touch rules/commit-convention.md
touch rules/architecture.md
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Path rules -Force
New-Item -ItemType File -Path rules/coding-style.md
New-Item -ItemType File -Path rules/commit-convention.md
New-Item -ItemType File -Path rules/architecture.md
```

활동 2-3 · 산출물: RULES/ 3개 파일 채워짐

rules/ AI에 채우게 하기 — 명령

CLAUDE.md 3영역(코딩 가이드라인)을 참고해 각 파일 50줄 이내로 작성.

"CLAUDE.md의 3영역(코딩 가이드라인)을 참고해서
rules/ 폴더의 3개 파일을 채워줘:

- rules/coding-style.md: 네이밍·포맷·들여쓰기 상세
- rules/commit-convention.md: 커밋 type 목록과 예시
- rules/architecture.md: 폴더 구조와 모듈 분리 원칙

각 파일은 50줄 이내로 짧게 작성해줘."

CLAUDE.md 마지막에 참조 지침 추가

docs/와 rules/ 6개 파일을 항상 참조하는 헌법 조항 — 디자인 가이드는 다음 단계.

5. 시스템 설계 참조 지침

다음 문서를 항상 참조한다:

- docs/prd.md: 제품 요구사항
- docs/persona.md: 페르소나 가설
- docs/painpoints.md: 검증된 페인포인트
- rules/coding-style.md: 네이밍 포맷
- rules/commit-convention.md: 커밋 형식
- rules/architecture.md: 폴더 구조 원칙

기능 구현 시 docs/prd.md의 핵심 기능 3개에 매핑되는지 확인.
모호함이 발견되면 임의 해석하지 말고 작업자에게 확인 요청.

(디자인 가이드는 다음 단계에서 추가됨)

STEP 2 · 검증 / 성공 기준

Step 3 진입 가능한가

4개 항목 모두 통과 시 Step 3 (settings.json + Permissions)로 진입.

- 01

☒

docs/ 폴더가 PRD 기반 문서 3개를 담고 있는가
- 02

☒

rules/ 폴더가 3개 파일로 구성됐는가
- 03

☒

CLAUDE.md에 시스템 설계 참조 지침이 명시됐는가
- 04

☒

디자인 가이드는 다음 단계로 미뤄졌는가

— STEP 3

settings.json과 Permissions

권장 템플릿 적용 + 의도된 차단 확인

- 01 3-1. .claude/settings.json 생성
- 02 3-2. 권장 템플릿 입력 (auto_allow / ask_before / deny)
- 03 3-3. 의도된 차단 시연 (/etc/hosts 차단 확인)

활동 3-1 · 산출물: .CLAUDE/SETTINGS.JSON 빈 파일

.claude/settings.json 생성

프로젝트별 Claude Code 설정 — Permissions 3단계가 들어갈 파일.

MACOS / LINUX

```
mkdir -p .claude
touch .claude/settings.json
```

WINDOWS POWERSHELL

```
New-Item -ItemType Directory -Path .claude -Force
New-Item -ItemType File -Path .claude/settings.json
```


활동 3-2 · 산출물: SETTINGS.JSON 권장 템플릿 적용

권장 템플릿 입력 (그대로)

그대로 적용하면 안전과 속도의 균형이 자동으로 갖춰진다.

파일 경로 **.claude/settings.json**

```
{
  "permissions": {
    "allow": ["Read", "Grep", "Glob", "Edit"],
    "ask": ["Bash", "Write", "WebFetch"],
    "deny": ["BashWithSudo"]
  }
}
```

— 활동 3-3 · 산출물: 의도된 차단 통과 확인

의도된 차단 시연 — deny 동작 확인

시스템 파일 수정 명령으로 deny 동작이 작동하는지 확인.

```
"시스템 파일 /etc/hosts를 수정해줘"
```

통과 기준
차단되면 정상 — BashWithSudo가 deny에 들어가 있어 권한 자체 차단됨.

STEP 3 · 검증 / 성공 기준

Step 4 진입 가능한가

3개 항목 모두 통과 시 Step 4 (GitHub 셋업)로 진입.

- 01

 .claude/settings.json이 생성됐는가
- 02

 Permissions 3단계가 명시됐는가
- 03

 의도된 차단이 동작했는가

— STEP 4

GitHub 저장소 셋업과 초기 커밋

planning_project repo + 초기 셋업 첫 커밋

- 01 4-1. GitHub 저장소 생성 (planning_project)
- 02 4-2. 로컬과 원격 연결 (git remote add)
- 03 4-3. .gitignore 생성 (Next.js 기준, .claude/ 포함)
- 04 4-4. 초기 셋업 커밋 (자연어 요청)
- 05 4-5. 커밋 확인 점검

GitHub 저장소 생성 — 메뉴 조작 6 STEP

repo 이름도 planning_project 일치 — 폴더와 저장소가 같은 이름.

- 1 github.com 접속
- 2 우측 상단 "+" → "New repository"
- 3 Repository name: planning_project
- 4 Privacy: Private (또는 Public)
- 5 "Create repository" 클릭
- 6 URL 복사 — 다음 단계에서 git remote add origin에 사용

— 활동 4-2 · 산출물: 원격 ORIGIN 등록 완료

로컬과 원격 연결

{user}는 본인 GitHub 사용자 이름으로 치환.

공통 명령 (MACOS / LINUX / WINDOWS)

```
git remote add origin https://github.com/{user}/planning_project.git  
git branch -M main
```

— 활동 4-3 · 산출물: .GITIGNORE 파일

.gitignore 생성 — 자연어 요청

Next.js 기준. .claude/는 제외하지 말고 포함 — 팀 단위 헌법.

"이 프로젝트에 맞는 .gitignore를 만들어줘.
Next.js 기준이고, .claude/는 제외하지 말고 포함.
node_modules, .env, build 결과물은 제외."

초기 셋업 커밋 — 자연어 요청 + 시드 6 메시지

커밋 메시지는 CLAUDE.md 3영역 형식([type] description)을 따른다.

자연어 커밋 요청

"현재까지의 초기 셋업을 GitHub에 커밋해줘.
커밋 메시지는 CLAUDE.md 3영역의 형식을 따라줘.
단, 변경 파일을 먼저 보여주고
내 확인을 받은 뒤 진행."

시드 6번 예상 커밋 메시지

[chore] 초기 셋업: CLAUDE.md, docs/, rules/, settings.json

- CLAUDE.md 4영역 + 시스템 설계 참조 지침
- docs/prd.md, persona.md, painpoints.md
- rules/coding-style.md, commit-convention.md, architecture.md
- .claude/settings.json (Permissions 3단계)
- .gitignore

— 활동 4-5 · 산출물: GITHUB 저장소 4 항목 통과

커밋 확인 점검

GitHub 웹에서 직접 확인 — 4개 항목 모두 통과.

- OK

CLAUDE.md 저장소에 보임
- OK

docs/, rules/ 폴더 보임
- OK

.claude/settings.json 보임
- OK

커밋 메시지가 [type] description 형식

STEP 4 · 검증 / 성공 기준

Step N (정리·회고) 진입 가능한가

4개 항목 모두 통과 시 Step N으로 진입.

01 ☒ GitHub 저장소가 planning_project 이름으로 생성됐는가

02 ☒ 로컬과 원격이 연결됐는가

03 ☒ 초기 셋업이 첫 커밋으로 푸시됐는가

04 ☒ 커밋 메시지가 CLAUDE.md 형식을 따르는가

— STEP N

정리와 비용 비교

산출물 확인, 토큰 비교, 회고

- 01 N-1. 결과 정리 (산출물 6개 매핑)
- 02 N-2. /cost 비교 (시작 → 종료 토큰 차이)
- 03 N-3. 회고 3 질문 (좋은 답 vs 나쁜 답)

결과 정리 — 산출물 × 다음 단계 활용처

6개 산출물이 모두 손에 있어야 다음 세션(자료 5)으로 자신 있게 넘어간다.

1 CLAUDE.md (검증 통과) 활용 자동 컨텍스트	2 .claude/settings.json 활용 Permissions	3 docs/ (3개 파일) 활용 시스템 설계 참조
4 rules/ (3개 파일) 활용 코딩 규칙	5 .gitignore 활용 버전 관리	6 GitHub 저장소 + 초기 커밋 활용 다음 단계 입력

— 활동 N-2 · 산출물: 토큰 차이 메모

/cost 비교 — 시작 vs 종료

Step 0의 베이스라인 토큰과 비교. 차이가 본 세션의 실비용이다.

CLAUDE CODE 세션 안에서

/cost

메모 항목

- 시작 토큰 (Step 0 메모)
- 종료 토큰 (현재)
- 차이 — 본 세션 누적 비용 (USD)
- 세션 시간 (시작 ~ 종료)

회고 좋은 답 vs 나쁜 답

같은 질문에 답이 측정 가능·구체적이면 좋은 답.

질문	좋은 답	나쁜 답
AI 검증 효과	"1차 초안 페르소나 모호하다고 AI가 짚어줘 1인 셀러로 명확화"	"더 좋아진 것 같다"
가장 모호했던 영역	"1영역 성공 지표가 측정 단위 누락이었음"	"전반적으로 어려웠다"
다음 단계 준비	"화면 분석과 Task 재구성으로 진입 가능"	"준비됐다"

STEP N · 검증 / S4 완주 기준

S4 완주 — 다음 세션 진입 가능한가

4개 항목 모두 통과 시 S4 종료. 다음 세션 자료 5(화면 분석 + Task 재구성)로 진입.

- 01

☒

산출물 6개가 모두 손에 있는가
- 02

☒

/cost 비교를 메모했는가
- 03

☒

회고 3개 질문에 모두 답했는가
- 04

☒

다음 단계 화면 분석 진입 준비가 됐는가

본 자료의 과제

다음 단계(자료 5)에서 본 산출물을 입력으로 받아 화면 분석과 Task 재구성을 진행한다.

항목	내용
목표	빌드 진입 가능한 상태로 환경 안정화
산출물	CLAUDE.md, docs/, rules/, settings.json, GitHub 저장소 초기 커밋
마감	다음 학습 단계 시작 전

— TIPS 1 · 폴더와 저장소 일관성

학습 전체가 하나의 폴더에서 누적

[PLANNING_PROJECT](#)로 고정

학습 단계가 진행되어도 같은 폴더에서 누적. 매 단계마다 새 폴더로 옮기면 이전 산출물 다시 복사해야 하고 흐름이 끊긴다.

저장소 이름도 폴더와 일치

GitHub 저장소 이름도 `planning_project`. 학습 전체가 하나의 저장소에서 진행.

헌법은 거칠게 시작해 협업으로 완성한다

PRD와 CLAUDE.md의 차이

PRD는 무엇을, CLAUDE.md는 어떻게. 둘이 헛갈리면 자료가 부풀거나 모호해진다.

1차 초안은 거칠어도 된다

작업자가 의도를 거칠게 담는 게 1차 목적. AI가 개선·보강하는 협업 흐름. 완벽한 초안 만들려고 막히지 말 것.

200줄 미만 유지

매 명령마다 자동 로드되는 토큰. 4영역 모두 채우면서 30-50줄이 균형.

— TIPS 3 · AI 검증 루프 팁

AI 답변이 모호 신호를 알려준다

AI 답변이 매번 다르면 모호 신호

AI가 일관된 답을 못 한다는 건 컨텍스트가 흐릿하다는 뜻. "어느 영역이 가장 모호한가"를 AI에 직접 질문.

답변이 의도와 다르면 못 짚은 부분이 모호

AI가 페르소나를 못 짚으면 1영역 모호. 핵심 기능 못 짚으면 핵심 기능 영역 모호.

Permissions = 자율과 안전의 균형점

PERMISSIONS는 단순 차단이 아님

OODA 루프의 Decide→Act 사이 작업자 개입 지점.
도구 위험도에 따라 자율 / 확인 / 금지 분배.

권장 템플릿을 그대로 사용

자유 변형은 자가 학습 영역.
본 학습은 템플릿 그대로 사용으로 안전과 속도 균형.

Issue는 다음 단계로, 커밋은 자연어로

ISSUE 등록은 다음 단계로

본 단계에서 Task를 Issue로 등록하지 않음.
화면 분석 후 그룹화된 Task로 Issue 등록이 자연스러운 흐름.

자연어 커밋

git 명령 직접 입력 안 함.
"현재 셋업을 커밋해줘"로 운영. CLAUDE.md 규칙을 따른 커밋 메시지 자동 생성.

— TIPS 6 · 시스템 환경 팁

본 단계의 경계선과 헌법 위반 지적법

DOCS/와 RULES/는 필수, 디자인 가이드는 다음 단계

현 단계: docs/(PRD 참조)와 rules/(코딩 규칙)만.

디자인 가이드는 다음 단계 빌드 시작 시 추가.

헌법 위반은 조항 번호로 지적

"CLAUDE.md 3조의 커밋 메시지 형식을 어겼다"라고 명시.

에이전트가 정확히 인식.